📘 سند رسمی فنی — نسخه ۱.۴.۰

برچسب‌خوان هوشمند دارو و محصولات

LabelVoice

معماری سامانه، نحوه کارکرد، معرفی تک‌تک فایل‌ها و همه کتابخانه‌ها و فناوری‌های به‌کاررفته — سامانه‌ای دو زیرسیستمی (دسکتاپ + موبایل) که فقط «متن روی بسته» را می‌خواند و پاسخ فارسی ایمن تولید می‌کند.

👨‍💻 توسعه و طراحی: **حمید جلیلوند** و **نوید بدری**

۲	+۱۲۷	۱۴۹	۱۰۰٪
زیرسیستم هم‌منطق	فایل پروژه	آزمون خودکار	پردازش محلی

■ بخش ۱

نمای کلی محصول

برچسب‌خوان هوشمند (LabelVoice) سامانه‌ای است برای خواندن متن روی بسته‌بندی دارو و محصولات با فناوری OCR و تبدیل آن به پاسخ فارسی مرتب، ساخت‌یافته و قابل پخش صوتی. محصول دو زیرسیستم هم‌منطق دارد: یک نسخه دسکتاپ با پایتون و Streamlit، و یک نسخه موبایل به‌شکل PWA قابل نصب که همه پردازش را روی خود گوشی انجام می‌دهد. مرز این ابزار روشن و تغییرناپذیر است: فقط می‌گوید «روی بسته چه نوشته شده» — نه یک کلمه بیشتر. هیچ توصیه پزشکی، کاربرد دارو، مقدار مصرف یا تشخیصی ارائه نمی‌شود و اطلاعات غایب از اینترنت یا حافظه تکمیل نمی‌گردد.

🧠 رویکرد مبتنی بر شواهد

پاسخ از قواعد و قالب‌های فارسی ساخته می‌شود — بدون هیچ مدل زبانی ابری در حلقه. هر عدد داخل پاسخ باید در شواهد OCR موجود باشد؛ بنابراین «توهم» پاسخ در این معماری ممکن نیست و هر جمله قابل ردیابی به تصویر است.

⛔ خطوط قرمز ایمنی

عبارت «قدرت درج‌شده» جایگزین اجباری «دوز مصرف» است؛ نگهبان ایمنی واژه‌های ممنوع را رد می‌کند، اعداد خارج از شواهد را مسدود می‌کند و ادعای کاربرد، تشخیص یا توصیه پزشکی را همیشه می‌بندد.

■ بخش ۲

معماری سامانه

دو زیرسیستم مستقل با منطق یکسان طراحی شده‌اند تا تجربه و رفتار محصول روی رایانه و گوشی یکسان باشد؛ هر دو دقیقاً همان قواعد نرمال‌سازی، استخراج، تناقض‌یابی و نگهبان ایمنی را پیاده می‌کنند و خروجی JSON هم‌ساختار تولید می‌کنند. این «تطبیق دوقلو» با آزمون‌های موازی (pytest در برابر Node) تضمین می‌شود.

🖥️ زیرسیستم دسکتاپ — Python

رابط Streamlit با RTL کامل؛ پیش‌پردازش با OpenCV (CLAHE، حذف نویز، تیزکردن، صاف‌کردن چرخش، سه نسخه خروجی)؛ OCR با Tesseract 5 (PSM 3+6 و ادغام فازی) و EasyOCR اختیاری؛ مدل داده با Pydantic v2 و وضعیت پنج‌گانه فیلد؛ گفتار با edge-tts/gTTS فقط با رضایت صریح؛ ۷۹ آزمون pytest.

📱 **زیرسیستم موبایل / PWA** — JavaScript

HTML/CSS/JS خالص (ES5/IIFE) بدون چارچوب سنگین؛ دوربین زنده با getUserMedia (دوربین پشتی) و انتخاب تیزترین فریم از ۳ فریم؛ تشخیص جعبه دارو با سوبل و مورفولوژی؛ OCR با Tesseract.js 6 روی WebAssembly و داده زبان محلی fas+eng؛ گفتار با Web Speech API روی خود دستگاه؛ ۵۴ آزمون ۱۶ + Node آزمون E2E مرورگر.

🔗 **پل هم‌منطقی:** قواعد نرمال‌سازی نویسه، الگوهای regex، ادغام فازی (شباهت ≤ ۰٫۸)، محافظ مقدار صفر، تشخیص تناقض، نگهبان ایمنی و ساختار JSON — همگی جفتی (Python ⇄ JS) پیاده و با آزمون‌های هم‌راستا تأیید شده‌اند.

■ بخش ۳

ملاحظات کلیدی پروژه و محل پیاده‌سازی

تعریف مسئله: کاربر دوربین را به سمت جعبه دارو یا بسته‌بندی یک محصول می‌گیرد. سامانه نام، اطلاعات چاپ‌شده و سایر فیلدهای قابل خواندن را استخراج می‌کند و آن‌ها را به زبان فارسی نمایش و می‌خواند. برای مثال، روی بسته دارو نام، قدرت درج‌شده و تعداد قرص‌ها وجود دارد. اصل حاکم بر کل معماری این است که سامانه باید اطلاعات خوانده‌شده را از توصیه‌های پزشکی جدا نگه دارد: خروجی فقط بازگویی وفادار به متن چاپ‌شده است و هیچ کاربرد، مقدار مصرف، تشخیص یا توصیه‌ای تولید نمی‌شود.

نمودار معماری مرجع — از تصویر بسته‌بندی تا خوانش فارسی، با فایل پیاده‌سازی هر مرحله در دو زیرسیستم:

۰ **تصویر بسته‌بندی (ورودی)** دوربین / فایل

عکس جعبه دارو یا بسته محصول وارد و اعتبارسنجی می‌شود (magic bytes، حجم، ابعاد). **دسکتاپ:** app.py + validation.py — **موبایل:** js/camera.js + js/app.js

۱ **OpenCV — تشخیص ناحیه برچسب، اصلاح زاویه و بهبود وضوح** OpenCV · Canvas

تشخیص ناحیه برچسب و جعبه (لبه‌یابی + بزرگ‌ترین جزء همبند)، صاف‌کردن چرخش، مقیاس‌گذاری، CLAHE و تیزکردن. **دسکتاپ:** preprocess.py + quality.py — **موبایل:** js/boxdetect.js + js/preprocess.js

۲ **OCR — استخراج نام، اعداد و متن بسته‌بندی** Tesseract 5 · Tesseract.js (WASM)

خوانش چندنسخه‌ای و چند-PSM با ادغام فازی؛ خروجی قطعه‌های متنی با کادر و اطمینان. **دسکتاپ:** ocr.py — **موبایل:** js/ocr.js

۳ **پردازش متن و کنترل فیلدها** Regex دو زبانه · Pydantic

نرمال‌سازی نام و ارقام فارسی، استخراج قدرت و تعداد درج‌شده، گره‌زدن هر مقدار به شواهد و تشخیص تناقض. **دسکتاپ:** extractor.py + extraction/patterns.py + textnorm.py — **موبایل:** js/extractor.js + js/patterns.js + js/textnorm.js

۴ **تولید پاسخ فارسی — بازگویی وفادار به متن** موتور قاعده‌محور + نگهبان

پاسخ فقط از متن استخراج‌شده ساخته می‌شود (بدون LLM ابری) و از نگهبان ایمنی عبور اجباری دارد. **دسکتاپ:** response/builder.py + guard.py — **موبایل:** js/responder.js + js/guard.js

۵ **TTS + رابط کاربری — خواندن اطلاعات و نمایش متن استخراج‌شده** edge-tts · Web Speech API

نمایش ساخت‌یافته فیلدها و خواندن همان پاسخ تأییدشده. **دسکتاپ:** tts.py + app.py (Streamlit) — **موبایل:** speechSynthesis در js/app.js

ملاحظه کلیدی	فایل(ها) و نحوه پیاده‌سازی
جداسازی اطلاعات از توصیه پزشکی	نگهبان پاسخ — ممنوع‌واژه‌ها (از جمله «دوز مصرف»)، مسدودسازی ادعا، سلب مسئولیت اجباری: `response/guard.py` ⇄ `js/guard.js` (آزمون‌ها: test_response_builder.py، run_tests.js)
وفاداری به شواهد (بدون توهم)	هر فیلد به شناسه قطعه‌های OCR گره می‌خورد و هر عدد پاسخ باید در شواهد باشد: `extractor.py + builder.py` ⇄ `js/extractor.js + js/responder.js`
«قدرت درج‌شده» ≠ «دوز مصرف»	واژه‌گزینی اجباری در قالب‌ها + رد قطعی در نگهبان با آزمون اختصاصی: `builder.py + guard.py` ⇄ `js/responder.js + js/guard.js`
شفافیت ناخوانا/متناقض	وضعیت‌های پنج‌گانه فیلد + اصلاح دستی کاربر با تاریخچه: `models.py + ui_helpers.py` ⇄ `js/pipeline.js + js/app.js`
ورودی امن	بررسی امضای باینری PNG/JPEG/WebP، سقف حجم ۱۰MB و حداقل ابعاد: `validation.py` ⇄ `js/app.js`
کیفیت تصویر پیش از OCR	تاری (واریانس لاپلاسین)، تاریکی و انعکاس با هشدار شفاف: `quality.py` ⇄ `js/app.js`

■ بخش ۴

چگونه کار می‌کند؟ — خط پردازش ۹ مرحله‌ای

این خط در هر دو زیرسیستم با همان ترتیب اجرا می‌شود؛ تفاوت فقط در پیاده‌سازی (OpenCV ⇄ Canvas) است، نه در قواعد.

۱ **ورود و اعتبارسنجی تصویر** File API · magic bytes

بررسی امضای باینری PNG/JPEG/WebP، سقف حجم ۱۰ مگابایت و حداقل ابعاد — فایل غیرتصویری رد می‌شود تا موتور OCR با ورودی مخرب روبه‌رو نشود.

۲ **ارزیابی کیفیت** واریانس لاپلاسین · هیستوگرام

تاری، تاریکی و انعکاس نور سنجیده می‌شود و هشدارها به‌صورت چیپ رنگی نمایش داده می‌شوند تا کاربر در صورت نیاز عکس بهتری بگیرد.

۳ **پیش‌پردازش** OpenCV · Canvas

مقیاس هوشمند تا ۱۲۰۰ پیکسل کوتاه‌ترین ضلع، CLAHE (دسکتاپ)، تیزکردن Unsharp، صاف‌کردن چرخش ±۱۵ درجه و ساخت نسخه‌های اصلی، تیزشده و دودویی.

۴ **تشخیص و برش جعبه دارو** سوبل + مورفولوژی

ناحیه بسته با لبه‌یابی سوبل، آستانه درصدی، اتساع و بزرگ‌ترین جزء همبند پیدا و از پس‌زمینه برش داده می‌شود؛ نویز OCR به‌شدت کاهش می‌یابد.

۵ **OCR چندنسخه‌ای و چند-PSM** Tesseract 5 · Tesseract.js (WASM LSTM)

هر نسخه تصویر با دو حالت چیدمان (PSM 3 و PSM 6 و در موبایل PSM 11 پراکنده) خوانده می‌شود؛ user_defined_dpi برای دقت تنظیم شده است.

۶ **ادغام فازی خوانش‌ها** شباهت ترتیبی ≥ ۰٫۸

خوانش‌های مشابه ادغام و تکرارها حذف می‌شوند؛ بازده خوانش امتیازدهی می‌شود و در خوانش ضعیف، چرخش‌های ۹۰ درجه به‌طور خودکار آزموده می‌شوند.

۷ **استخراج فیلدها و تناقض‌یابی** Regex دو زبانه · Pydantic

نام، قدرت درج‌شده، شکل دارویی، حجم/وزن، تاریخ، سری و سازنده استخراج می‌شود؛ هر مقدار به شناسه قطعه‌های شاهد گره می‌خورد و خوانش‌های متناقض شفاف اعلام می‌شوند.

۸ **تولید پاسخ فارسی** موتور قاعده‌محور

مقدمه بر اساس نوع بسته (دارو یا محصول) انتخاب و پاسخ از قالب‌های فارسی ساخته می‌شود — بدون هیچ مدل زبانی ابری و بدون تکمیل اطلاعات از حافظه.

۹ **نگهبان ایمنی و خروجی** Guard · TTS · JSON

واژه‌های ممنوع رد، اعداد خارج از شواهد مسدود و سلب مسئولیت تزریق می‌شود؛ سپس نمایش، پخش صوتی و دانلود JSON انجام می‌گیرد.

■ بخش ۵

فایل‌های سامانه — مرجع کامل

تک‌تک فایل‌های مخزن به تفکیک دسته، همراه با نقش هر فایل. پوشه‌های داده حجیم (مدل‌ها، نمونه‌ها، آیکون‌ها) به‌صورت گروهی معرفی شده‌اند.

فایل (هسته پایتون — label_reader/)	نقش
`__init__.py`	متادیتا، نام فارسی محصول (APP_NAME_FA)، محدوده (SCOPE_FA) و نسخه
`config.py`	تنظیمات با pydantic-settings: موتور OCR، آستانه‌های کیفیت، ابعاد مقیاس، سقف حجم
`models.py`	مدل‌های Pydantic v2: OcrPiece، RawLine، Field (وضعیت پنج‌گانه + شواهد)، Metadata

فایل (هسته پایتون — label_reader/)	نقش
textnorm.py	نرمال‌سازی نویسه: ي→ی، ك→ک، ارقام فارسی/عربی به لاتین، یکسان‌سازی فاصله
validation.py	اعتبارسنجی: magic bytes، سقف حجم/ابعاد، خطای ImageValidationError با پیام فارسی
quality.py	تاری (لاپلاسین)، تاریکی، انعکاس؛ هیوریستیک پس‌زمینه‌آگاه و سطوح شدت
preprocess.py	OpenCV: مقیاس هوشمند، CLAHE، حذف نویز، تیزکردن، deskew، سه نسخه خروجی
pipeline.py	هماهنگ‌کننده کل خط: اعتبارسنجی تا پاسخ و نگهبان
ui_helpers.py	کمک‌های رابط: RTL، جدول فیلدها، رسم کادرهای OCR، برچسب‌های فارسی
extraction/classify.py	تشخیص نوع بسته: دارو یا محصول عمومی بر اساس واژگان و الگوها
extraction/patterns.py	الگوهای regex دو زبانه: قدرت (mg/µg/mL)، شکل دارویی، تاریخ، سری، سازنده
extraction/fields.py	نقشه‌برداری قطعه‌ها به فیلدها + ساخت Field با شواهد و یادداشت فارسی
extraction/conflicts.py	تشخیص تناقض میان خوانش‌ها با محافظ مقدار صفر
ocr/tesseract_provider.py	Tesseract 5 با pytesseract: دو PSM، ادغام فازی ≤ ۰٫۸، کادر و اطمینان
ocr/easyocr_provider.py	EasyOCR fa+en — بارگیری lazy برای تصاویر سخت
response/builder.py	قالب‌های فارسی پاسخ بر اساس نوع بسته و فیلدهای موجود
response/guard.py	نگهبان ایمنی: واژه‌های ممنوع، نظارت اعداد خارج از شواهد، سلب مسئولیت
response/tts_prep.py	آماده‌سازی متن برای تلفظ (اعداد و علائم) بدون تغییر معنا
tts/edge_provider.py · gtts_provider.py	edge-tts (صداهای عصبی فارسی) و gTTS با پیام وضعیت فارسی
llm/rewriter.py	اسکلت بازنویسی اختیاری با LLM — پیش‌فرض خاموش؛ نگهبان همیشه فعال

فایل (نسخه موبایل — mobile-app/)	نقش
index.html	پوسته برنامه: هدر، دکمه‌ها، تنظیمات، نتیجه، اورلی دوربین زنده + دکمه‌های راهنما
css/style.css	استایل موبایل‌محور RTL، تم روشن/تاریک خودکار + استایل مودال‌های راهنما
manifest.webmanifest · sw.js	مانیفست PWA و Service Worker cache-first برای اجرای آفلاین کامل
js/textnorm.js · patterns.js · guard.js	پورت نرمال‌سازی، الگوها و نگهبان ایمنی (یکسان با نسخه پایتون)

فایل (نسخه موبایل — mobile-app/)	نقش
`js/extractor.js · responder.js`	استخراج فیلدها + تشخیص تناقض + قالب‌های پاسخ فارسی و مقدمه نوع محصول
`js/pipeline.js`	ارزیابی کیفیت، امتیاز بازده OCR، ساخت نتیجه، اصلاح کاربر
`js/preprocess.js · boxdetect.js`	پردازش Canvas: مقیاس، تیزکردن، دودویی، deskew، چرخش، برش + تشخیص جعبه
`js/ocr.js`	پوشش Tesseract.js: نسخه‌ها، دو PSM، PSM 11، ادغام فازی، گزارش پیشرفت
`js/camera.js`	دوربین زنده getUserMedia: فلاش، زوم، تعویض دوربین، ۳ فریم و انتخاب تیزترین
`js/help.js (ارزیابی + ۱.۳.۰ ۱.۴.۰)`	مودال‌های راهنمای درون‌برنامه‌ای + معرفی فناوری هر بخش + مودال ارزیابی مدل‌ها
`js/app.js`	کنترل‌گر رابط: رویدادها، پیش‌نمایش، خط پردازش، نتیجه، JSON، TTS
`js/samples-data.js`	نمونه‌های base64 تعبیه‌شده برای آزمایش آفلاین
`label-reader-singlefile.html`	نسخه تک‌فایلی (حدود ۴۰۰ کیلوبایت) — اجرای با دابل‌کلیک بدون سرور
`documentation.html (۱.۳.۰ + ارزیابی ۱.۴.۰)`	مستندات فنی مصور و رسپانسیو (نسخه HTML این سند) + بخش ملاحظات و ارزیابی با نمودار
`docs/evaluation/evaluation-results.json (جدید ۱.۴.۰)`	نتایج کامل ارزیابی (دقت فیلد/حالت، زمان، آزمون‌ها) — منبع نمودارها
`vendor/ · lang/ · assets/fonts/ · icons/`	Tesseract.js 6 (WASM)، داده زبان fas+eng (integer و best)، فونت وزیرمتن و آیکون‌ها
`START-WINDOWS.bat · server-windows.ps1`	لانچر ویندوز ۱۰۰٪ ASCII + سرور TcpListener بدون پایتون (با حفاظت مسیر)

فایل (اندروید، آزمون، اسکریپت، مستندات)	نقش
`android/MainActivity.java`	WebView با مجوز onPermissionRequest، CAMERA و پشتیبانی دوربین زنده
`android/AndroidManifest.xml · build_apk.sh`	مانیفست و اسکریپت ساخت APK (نیازمند JDK + SDK روی ماشین کاربر)
`tests/ (۸ فایل pytest)`	۷۹ آزمون: نرمال‌سازی، الگوها، پاسخ، امنیت، کیفیت، TTS، یکپارچگی
`tests-js/run_tests.js`	۵۴ آزمون Node موازی با pytest (نگهبان‌ها + جعبه + بازده)
`scripts/make_sample_images.py`	ساخت ۱۴ برچسب نمونه (فارسی/انگلیسی/مخرب/مبهم) با فونت وزیرمتن

فایل (اندروید، آزمون، اسکریپت، مستندات)	نقش
`scripts/evaluate.py`	سنجش دقت فیلد روی مجموعه نمونه (۸۵٫۷٪ با Tesseract)
`scripts/run_demo_cli.py` · `download_tessdata.py` · `build_singlefile.py`	دموی خطی، دانلود داده زبان، ساخت نسخه تک‌فایلی
`docs/ARCHITECTURE.md` · `INSTALL.md` · `LIMITATIONS.md`	معماری متنی، نصب گام‌به‌گام، محدودیت‌های شناخته‌شده
`models/tessdata/` · `sample_images/` · `assets/fonts/`	traineddata دسکتاپ، ۱۴ تصویر نمونه + manifest، فونت وزیرمتن

■ بخش ۶

کتابخانه‌ها، ابزارها و فناوری‌ها

همه اجزایی که سامانه روی آن‌ها ساخته شده، با نقش دقیق و محل استفاده:

- **Python 3.12** — زبان اصلی دسکتاپ و اسکریپت‌ها
- **OpenCV** — پیش‌پردازش تصویر (CLAHE، نویز، تیزی، deskew)
- **Tesseract 5 + pytesseract** — OCR دسکتاپ (LSTM، دو PSM)
- **EasyOCR** — مدل عمیق اختیاری fa+en
- **Pydantic v2** — مدل داده و تنظیمات با اعتبارسنجی
- **Streamlit** — رابط وب دسکتاپ RTL + مودال @st.dialog
- **Pillow** — باز/تبدیل تصویر و نمونه‌ساز (raqm برای فارسی)
- **edge-tts** — گفتار عصبی فارسی (فقط با رضایت)
- **gTTS** — جایگزین گفتار
- **Tesseract.js 6 (WASM)** — OCR مرورگر، کاملاً محلی
- **getUserMedia** — دوربین زنده پشتی (استاندارد W3C)
- **Canvas API** — پردازش پیکسلی موبایل
- **Web Speech API** — گفتار روی دستگاه (fa-IR)
- **PWA + Service Worker** — نصب و اجرای آفلاین
- **فونت وزیرمتن** — فونت فارسی متن‌باز، محلی و آفلاین
- **Android WebView** — پوسته APK با مجوز دوربین
- **pytest** — ۷۹ آزمون دسکتاپ
- **Node.js** — ۵۴ آزمون واحد موبایل + E2E مرورگر
- **pandas + Altair** — نمودارهای بخش ارزیابی در رابط دسکتاپ

■ بخش ۷

حریم خصوصی و خطوط ایمنی

🔒 پردازش صددرصد محلی

در هر دو زیرسیستم، OCR و استخراج روی دستگاه اجرا می‌شود. تنها استثنا، گفتار دسکتاپ است که فقط با رضایت صریح کاربر به سرویس TTS ارسال می‌شود؛ نسخه موبایل حتی آن را هم ندارد و گفتار روی خود گوشی انجام می‌شود.

🛡️ نگهبان پاسخ

پاسخ پیش از نمایش بازبینی می‌شود: واژه‌های ممنوع (از جمله «دوز مصرف») رد می‌شوند، هر عدد باید در شواهد OCR موجود باشد و ادعای کاربرد، تشخیص یا توصیه مسدود است؛ سلب مسئولیت به پاسخ تزریق می‌شود.

🗒️ پاسخ مبتنی بر شواهد

هر فیلد به شناسه قطعه‌های OCR گره خورده است؛ کاربر می‌بیند هر عدد از کدام سطر تصویر آمده و در صورت اشتباه، با حفظ تاریخچه آن را اصلاح می‌کند. «قدرت درج‌شده» همیشه جایگزین «دوز مصرف» است.

🚫 مقاومت در برابر ورودی مخرب

نمونه‌های مخرب آزمون نشان می‌دهند نگهبان، دستورهای درون تصویر (مانند «توصیه پزشکی بده») را اجرا نمی‌کند؛ magic bytes، سقف حجم و ابعاد هم در لایه ورودی قرار دارند.

■ بخش ۸

ارزیابی و آزمون مدل‌ها — نتایج واقعی و اصول سنجش

موضوع این سامانه دارو است و هر اشتباه می‌تواند آسیب‌زا باشد؛ بنابراین ارزیابی بر اساس اصول بنیادین و منطقی سنجش سامانه‌های حساس انجام شده است. **اصول شش‌گانه:** ۱) زمینه‌سنجی برچسب‌خورده (۱۴ تصویر با مقادیر واقعی هر فیلد در sample_images/manifest.json — هیچ عددی جعل نمی‌شود)؛ ۲) سنجش در سطح فیلد با تطبیق نرمال‌شده، نه «متن شبیه هم»؛ ۳) خطای ایمنی = خطا — غیبت و تعارض هم خطا شمرده می‌شوند و مقدار حدسی هرگز قطعه ارائه نمی‌شود؛ ۴) نگهبان‌های ضدخطر با آزمون اختصاصی («دوز مصرف» ممنوع، صفر رد، اعداد خارج از شواهد ممنوع، تزریق دستور پالایش می‌شود)؛ ۵) شفافیت کامل محدودیت‌ها؛ ۶) تکرارپذیری با python scripts/evaluate.py و ثبت نتایج در docs/evaluation/evaluation-results.json.

🎯 دقت کل فیلدها — ۸۵٫۷٪

۱۲ از ۱۴ فیلد با Tesseract 5 روی مجموعه نمونه برچسب‌خورده — مطابق خط پایه منتشرشده از v1.0.0. میانگین زمان خط لوله کامل: ۱٫۸۷ ثانیه (میانه ۱٫۸۲، بیشینه ۲٫۸).

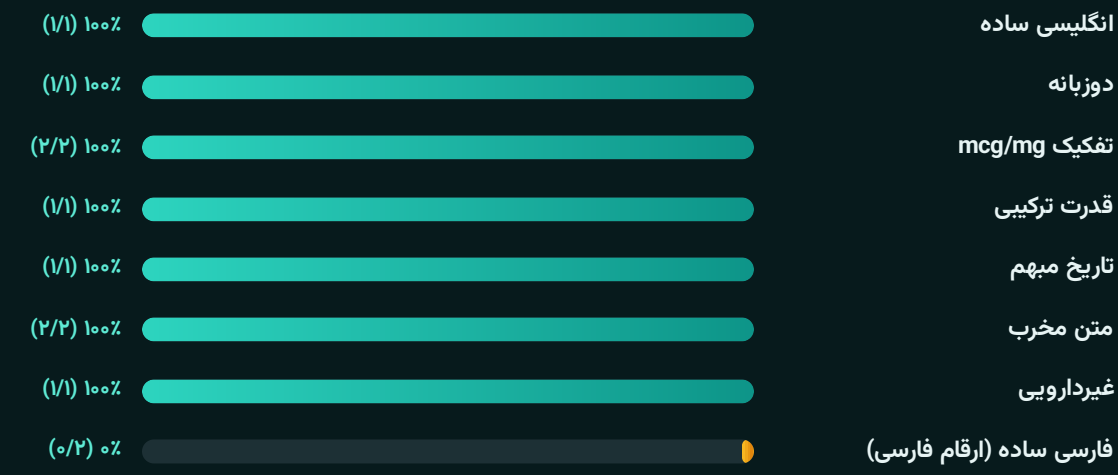
🧪 آزمون‌های خودکار — ۱۳۳ پاس

۷۹ آزمون pytest دسکتاپ + ۵۴ آزمون پاریتی Node موبایل — بدون هیچ شکست در اجرای این سند؛ نگهبان‌های ایمنی آزمون اختصاصی خود را دارند.

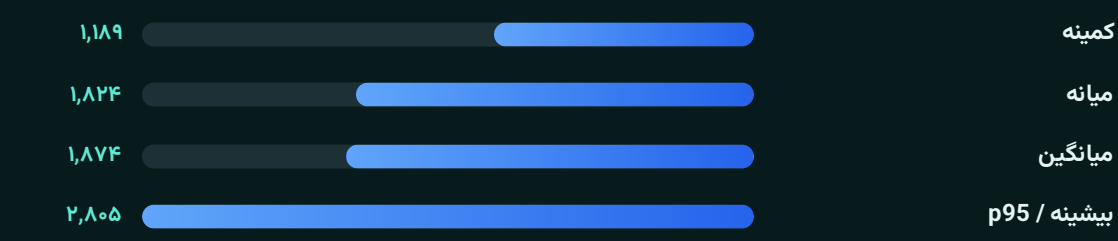
دقت به تفکیک فیلد (٪) — Tesseract 5:



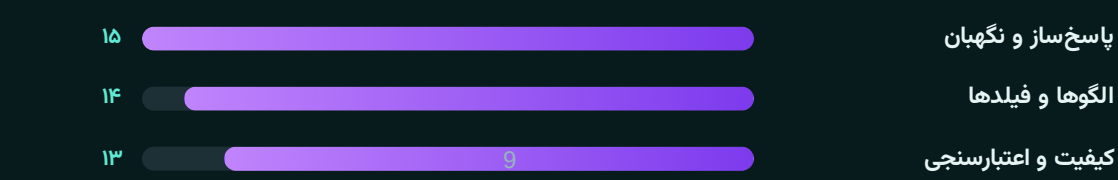
دقت به تفکیک حالت تصویر (٪):

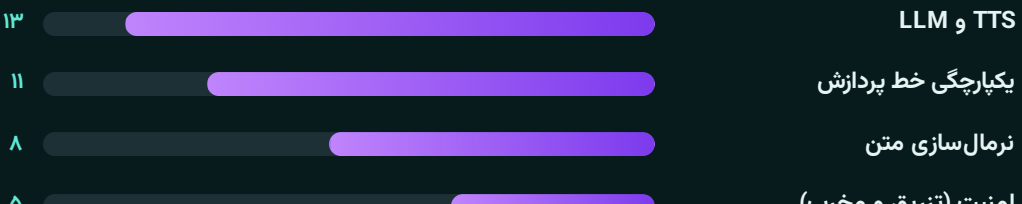


زمان پردازش هر تصویر (میلی‌ثانیه):



آزمون‌های واحد پایتون به تفکیک ماژول (۷۹ آزمون):





⚠️ یافته‌های صادقانه

ارقام فارسی: هر دو موتور OCR در ارقام فارسی ضعیف‌اند (حالت «فارسی ساده» ۰٪) — چنین مقادیری «نامطمئن» علامت می‌خورند و اصلاح دستی ارائه می‌شود؛ هیچ مقدار حدسی قطعه ارائه نمی‌شود. **EasyOCR:** در محیط اجرای این سند نصب نبود؛ انتخاب صریح آن با پیام شفاف OcrUnavailableError رد می‌شود — شکست شفاف، نه سکوت. **محل بازتولید:** docs/evaluation، tests-js/run_tests.js، tests/، /scripts/evaluate.py — نمودارهای زنده در رابط دسکتاپ (pandas + Altair) و مودال ارزیابی برنامه موبایل.

■ بخش ۹

محدودیت‌های شناخته‌شده

۰۹ ارقام فارسی در OCR

هر دو موتور در خواندن ارقام فارسی ضعیف‌اند (مثلاً ۵۰۰ گاهی ۰ خوانده می‌شود). این محدودیت مستند شده و با «محافظ مقدار صفر»، وضعیت «نامطمئن» و اصلاح دستی کاربر مدیریت می‌شود؛ پاسخ هرگز عدد مشکوک را قطعی ارائه نمی‌کند.

📷 وابستگی به کیفیت عکس

تاری شدید، انعکاس و زاویه زیاد، دقت را می‌کاهد؛ سامانه این موارد را شفاف اعلام می‌کند و نسخه موبایل با دوربین زنده و انتخاب تیزترین فریم، تأثیر را کاهش داده است.

🧪 تأیید اصالت نیست · LLM پیش‌فرض خاموش

ابزار اصالت دارو را تأیید نمی‌کند و فقط متن ظاهری بسته را می‌خواند. بازنویسی با مدل زبانی به‌صورت اسکلت آمده و پیش‌فرض خاموش است؛ حتی اگر فعال شود، خروجی دوباره از نگهبان عبور می‌کند.

■ بخش ۱۰

نصب و اجرا

🖥️ دسکتاپ

پس از نصب Python و Tesseract (با زبان fas): pip install -r requirements.txt سپس streamlit run app.py.
جزئیات کامل در docs/INSTALL.md.

📱 موبایل (PWA) و اندروید (APK)

آدرس را در مرورگر گوشی باز کنید و «افزودن به صفحه اصلی» را بزنید — اپ نصب و آفلاین اجرا می‌شود. برای APK: اسکریپت android/build_apk.sh با JDK و Android SDK (راهنما: android/README-ANDROID-fa.md).

■ بخش ۱۱

تاریخچه نسخه‌ها

۵ **نویسندگان + بخش ارزیابی مدل‌ها + تقویت مستندات** ۲۰۲۵ — v1.4.0

نام «حمید جلیلوند و نوید بدری» در هدر هر دو سامانه و هر دو سند؛ بخش «ملاحظات کلیدی و محل پیاده‌سازی» با نگاشت فایل‌ها؛ بخش «ارزیابی و آزمون مدل‌ها» با اصول سنجش سامانه‌های حساس دارویی و نمودارهای واقعی در هر دو سند و رابط دسکتاپ؛ مودال ارزیابی در موبایل.

۴ **APK بازساخت‌شده از آخرین کد** ۲۰۲۵ — v1.3.1

فایل قابل نصب اندروید (versionCode 6) از کد v1.3.0 ساخته و در Release منتشر شد — شامل دوربین زنده و مودال‌های راهنما که در APK قدیمی v1.0.2 نبود.

۳ **مودال‌های راهنما + مستندات مصور + PDF** ۲۰۲۵ — v1.3.0

راهنمای مودالی در همه بخش‌های برنامه (موبایل و دسکتاپ) با معرفی فناوری هر بخش؛ سند فنی HTML رسپانسیو با انیمیشن و نسخه PDF شکیل.

۲ **دوربین زنده، تشخیص جعبه و کیفیت بالاتر OCR** ۲۰۲۵ — v1.2.0

دکمه دوربین به دوربین زنده واقعی (getUserMedia) تبدیل شد؛ تشخیص خودکار جعبه دارو، ۳ فریم و انتخاب تیزترین، PSM 11، تلاش چرخش ۹۰ درجه و پاسخ غنی‌تر؛ پشتیبانی اندروید.

۱ **رفع اشکال ویندوز و انتشار عمومی** v1.0.1 و v1.0.0

لانچرهای ۱۰۰٪ ASCII برای ویندوز؛ سپس انتشار اولیه با دسکتاپ کامل، PWA آفلاین، ۱۴۹ آزمون خودکار و انتشار روی GitHub Pages و Releases.

فقط متن روی بسته.

همین — و نه یک کلمه بیشتر.

برچسب‌خوان هوشمند (LabelVoice) ابزار خواندن برچسب است؛ نه سامانه تشخیص دارو، تجویز یا توصیه پزشکی. اطلاعات ناخوانا و متناقض به‌صورت شفاف اعلام می‌شوند و همه پردازش‌ها روی دستگاه شما انجام می‌شود.

سامانه: github.com/hjalilvand2004-coder/LabelVoice

نسخه وب: hjalilvand2004-coder.github.io/LabelVoice

سند HTML: …/LabelVoice/documentation.html